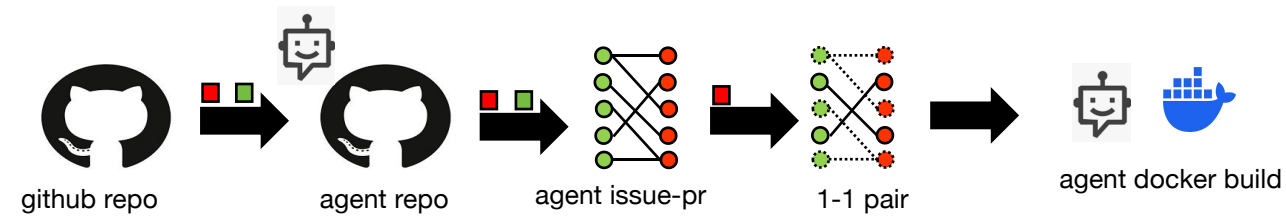
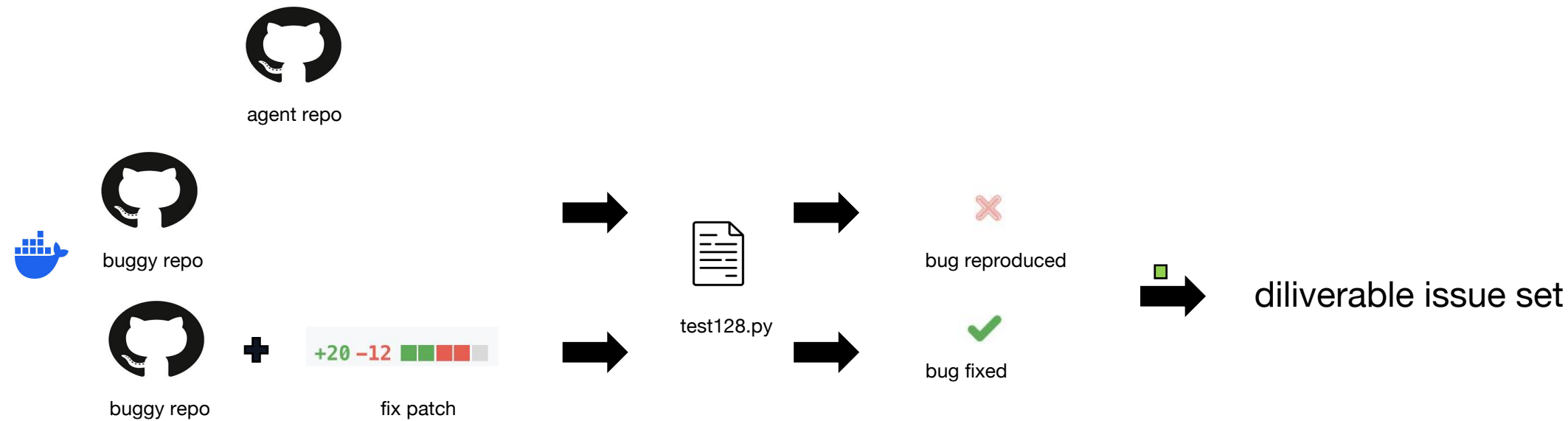
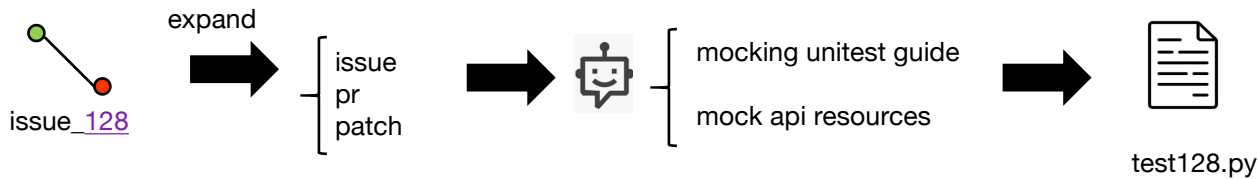


rule-based ai-based



previous



example issue 128

```
1  {
2    "number": 128,
3    "title": "Handle rate limit issues",
4    "url": "https://github.com/openai/codex/issues/128",
5    "state": "closed",
6    "created_at": "2025-04-16T22:20:04Z",
7    "closed_at": "2025-04-16T22:28:19Z",
8    "body": "Rate limit exceeded very fast for simple 'explain the project' prompt in a medium
9    size repository. A configuration option to avoid this would be nice.",
10   "labels": [],
11   "linked_prs": [
12     {
13       "number": 3617,
14       "state": "closed",
15       "title": "chore(deps): bump serde_json from 1.0.143 to 1.0.145 in /codex-rs",
16       "url": "https://github.com/openai/codex/pull/3617",
17       "merged": true,
18       "base_branch": "main"
19     },
20   ],
21   "ai_judgment": {
22     "is_agent_issue": true,
23     "response": "Yes. The issue involves LLM operation, specifically token usage
24     misconfiguration, as it mentions rate limits being exceeded, which is tied to LLM
25     provider usage and configuration."
26   }
27 }
```



example test 128

```
import unittest
from unittest.mock import patch, MagicMock
from openai import OpenAI
import time

class DummyAgent:
    """
    Dummy agent simulating interaction with the OpenAI/Forge API.
    Prior to the fix, it fails immediately on rate limit (no retry/backoff).
    After the fix, it retries/backoff according to configuration.
    """
    def __init__(self, api_key, base_url, max_retries=0, retry_wait=0.1):
        self.client = OpenAI(api_key=api_key, base_url=base_url)
        self.max_retries = max_retries
        self.retry_wait = retry_wait

    def explain_project(self, prompt="Explain the project"):
        attempts = 0
        while True:
            try:
                completion = self.client.chat.completions.create(
                    model="OpenAI/gpt-4o",
                    messages=[
                        {"role": "user", "content": prompt}
                    ]
                )
                return completion.choices[0].message.content
            except Exception as e:
                # Buggy version: no retry, immediately fail on rate limit
                if self.max_retries == 0: # Simulating buggy config
                    raise
                if "rate limit" in str(e).lower() and attempts < self.max_retries:
                    attempts += 1
                    time.sleep(self.retry_wait) # This sleep will be mocked
                else:
                    raise

class TestAgentRateLimitHandling(unittest.TestCase):
    """Test agent's handling of rate limit errors (issue #128)."""
    @patch("openai.OpenAI.chat.completions.create")
    @patch("time.sleep", return_value=None) # Patch sleep for speed
    def test_fails_fast_on_rate_limit_buggy(self, mock_sleep, mock_create):
        """
        Should raise immediately on rate limit (buggy version).
        """
        # Simulate API raising rate limit error instantly
        mock_create.side_effect = Exception("Rate limit exceeded")
        agent = DummyAgent(api_key="test-key", base_url="https://api.forge.tensorblock.co/v1", max_retries=0)
        with self.assertRaises(Exception) as cm:
            agent.explain_project()
        self.assertIn("rate limit", str(cm.exception).lower())
        mock_sleep.assert_not_called()
```

```
@patch("openai.OpenAI.chat.completions.create")
@patch("time.sleep", return_value=None)
def test_retries_and_recovers_after_fix(self, mock_sleep, mock_create):
    """
    Should retry up to max_retries and succeed if allowed (fixed version).
    """
    # First two calls raise, third one returns expected value
    success_msg = "This is the project explanation."
    mock_create.side_effect = [
        Exception("Rate limit exceeded"),
        Exception("Rate limit exceeded"),
        MagicMock(choices=[MagicMock(message=MagicMock(content=success_msg))])
    ]
    agent = DummyAgent(api_key="test-key", base_url="https://api.forge.tensorblock.co/v1", max_retries=2)
    result = agent.explain_project()
    self.assertEqual(result, success_msg)
    self.assertEqual(mock_create.call_count, 3)
    self.assertEqual(mock_sleep.call_count, 2) # slept after both rate limits
```

```
@patch("openai.OpenAI.chat.completions.create")
@patch("time.sleep", return_value=None)
def test_failure_after_max_retries(self, mock_sleep, mock_create):
    """
    Should raise after exceeding max_retries (fixed version edge case).
    """
    mock_create.side_effect = Exception("Rate limit exceeded")
    agent = DummyAgent(api_key="test-key", base_url="https://api.forge.tensorblock.co/v1", max_retries=3)
    with self.assertRaises(Exception) as cm:
        agent.explain_project()
    self.assertIn("rate limit", str(cm.exception).lower())
    self.assertEqual(mock_create.call_count, 4) # initial + 3 retries
    self.assertEqual(mock_sleep.call_count, 3)
```

```
if __name__ == "__main__":
    unittest.main()
```

example test 145

```
import unittest
from unittest.mock import patch, MagicMock
from openai import OpenAI

class DummyAgent:
    """
    Dummy agent for testing LLM provider switching (OpenAI/Ollama).
    Proper agent should honor base_url switching and not hardcode OpenAI usage or API key checks.
    """
    def __init__(self, api_key, base_url):
        self.client = OpenAI(api_key=api_key, base_url=base_url)

    def chat(self, prompt="What is 2+2?"):
        completion = self.client.chat.completions.create(
            model="llama2:7b-chat", # model string typical for Ollama
            messages=[{"role": "user", "content": prompt}]
        )
        return completion.choices[0].message.content

class TestAgentOllamaProvider(unittest.TestCase):
    """
    Test that the agent supports using an Ollama (OpenAI-compatible) endpoint.
    Verifies bug #145 (support for alternative providers via config, not just OpenAI hardcoding).
    """
    @patch("openai.OpenAI.chat.completions.create")
    def test_ollama_endpoint_via_base_url(self, mock_create):
        """
        Agent should work when base_url points to a local Ollama endpoint.
        This mocks the call, so it never hits a real Ollama/OpenAI backend.
        Should FAIL before patch if hardcoded to OpenAI, PASS after patch with base_url support.
        """
        ollama_url = "http://localhost:11434/v1"
        fake_response = MagicMock()
        fake_response.choices = [MagicMock(message=MagicMock(content="Ollama works!"))]
        mock_create.return_value = fake_response

        agent = DummyAgent(api_key="sk-placeholder", base_url=ollama_url)
        result = agent.chat(prompt="Ping Ollama")

        self.assertEqual(result, "Ollama works!")
        mock_create.assert_called_once()
        # Ensure the right parameters are passed to the client
        call_args = mock_create.call_args[1] if mock_create.call_args else {}
        self.assertIn("model", call_args)

    @patch("openai.OpenAI.chat.completions.create")
    def test_openai_endpoint_still_supported(self, mock_create):
        """
        Agent should continue working with the default OpenAI base_url.
        """
        openai_url = "https://api.openai.com/v1"
        fake_response = MagicMock()
        fake_response.choices = [MagicMock(message=MagicMock(content="OpenAI works!"))]
        mock_create.return_value = fake_response

        agent = DummyAgent(api_key="sk-testkey", base_url=openai_url)
        result = agent.chat(prompt="Ping OpenAI")

        self.assertEqual(result, "OpenAI works!")
        mock_create.assert_called_once()

if __name__ == "__main__":
    unittest.main()
```